

■ SESIÓN 2

Partes de un Sistema

Duración:	1 hora
Fase:	Fase 1 — Pensar antes de programar (Febrero)
Estado:	Material pedagógico preparado

■ 1. Idea central de la sesión

Al final de esta hora, debe quedar claro:

"Un sistema no es un bloque gigante, sino partes conectadas. Cada parte tiene un trabajo específico, y lo importante es saber QUÉ hace cada una y CÓMO se comunican entre ellas."

Por qué esto importa: Muchos principiantes (y muchas IAs) crean código donde todo está mezclado. Entender que un sistema tiene partes con responsabilidades claras es la base de la ingeniería de software real.

■ 2. Conexión con conocimientos previos

Desde Scratch:

En Scratch ya trabajaste con 'objetos' (sprites) que tienen sus propias responsabilidades:

- El personaje se mueve
- El enemigo persigue
- El marcador cuenta puntos

Diferencia clave:

- En Scratch: cada sprite es independiente
- En una app: las partes están **conectadas** y deben **comunicarse**

Desde la Sesión 1:

Ya dibujaste el sistema con cajas y flechas. Ahora vamos a **darle contenido a esas cajas**: qué información guarda cada una y cómo hablan entre ellas.

■ 3. Explicación clara del concepto

¿Qué es una 'parte' de un sistema?

Una parte es un componente que:

1. **Tiene una responsabilidad específica** (hace UNA cosa bien)
2. **Guarda información relevante** (su 'estado')
3. **Se comunica con otras partes** cuando es necesario

Ejemplo del mundo real:

Un coche tiene partes:

- **Motor:** genera movimiento (NO controla la radio)
- **Volante:** cambia dirección (NO enciende las luces)
- **Radio:** reproduce música (NO acelera el coche)

Si todo estuviera mezclado, arreglar la radio requeriría tocar el motor. **Eso es un mal diseño.**

Las 3 preguntas clave para cada parte

Para cada parte del sistema, siempre pregúntate:

1. **¿Qué hace?** (responsabilidad)
2. **¿Qué información necesita recordar?** (estado)
3. **¿Con quién habla?** (comunicación)

■ 4. Diseño antes de código

Partes de nuestra app (profundizando el dibujo de la Sesión 1)

■ Menú Principal

¿Qué hace?	• Muestra los juegos disponibles • Permite elegir uno • Cambia a la pantalla del juego seleccionado
¿Qué información guarda?	• Lista de juegos disponibles • Cuál está seleccionado (hover/foco)
¿Con quién habla?	• Sistema de navegación (para cambiar de pantalla)
¿Qué NO hace?	■ NO ejecuta los juegos ■ NO guarda puntuaciones ■ NO tiene lógica de juego

■ Motor de Juegos (Sistema Común)

¿Qué hace?	• Gestiona cosas comunes a todos los juegos: - Botón "volver al menú" - Mostrar puntuación - Reiniciar juego - Detectar victoria/derrota
¿Qué información guarda?	• Puntuación actual • Estado del juego (jugando/ganado/perdido) • Configuración común
¿Con quién habla?	• Cada juego específico (recibe eventos: "gané", "perdí") • Sistema de navegación (para volver al menú)
¿Qué NO hace?	■ NO conoce las reglas específicas de cada juego ■ NO dibuja los elementos del juego (solo el marco común)

■ Juego Específico (ej: Tic Tac Toe)

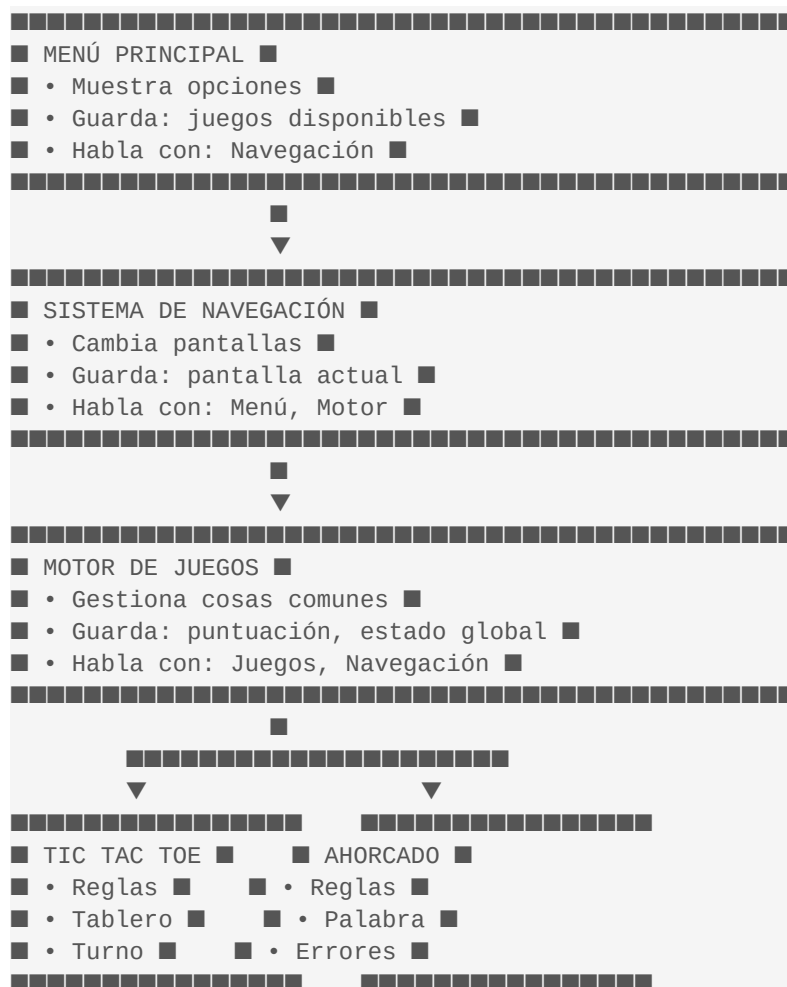
¿Qué hace?	• Implementa las reglas del juego • Valida movimientos • Detecta victoria/empate • Actualiza el tablero
------------	---

¿Qué información guarda?	• Estado del tablero (qué casilla tiene X, O, o vacía) • De quién es el turno • Número de movimientos
¿Con quién habla?	• Motor de juegos (le avisa: "se ganó", "movimiento válido") • Sistema de UI (para actualizar la pantalla)
¿Qué NO hace?	■ NO sabe que existe un menú ■ NO sabe que existen otros juegos ■ NO gestiona la navegación

■ Sistema de Navegación

¿Qué hace?	• Cambia entre pantallas (menú ↔ juego) • Oculta/muestra partes de la UI
¿Qué información guarda?	• Pantalla actual • Historial (para "volver atrás")
¿Con quién habla?	• Menú (recibe: "ir a juego X") • Motor de juegos (recibe: "volver al menú")
¿Qué NO hace?	■ NO tiene lógica de juegos ■ NO dibuja elementos de juegos

Diagrama de responsabilidades



■ 5. Ejemplo aplicado al proyecto

Caso concreto: 'Empezar una partida de Tic Tac Toe'

Veamos cómo **colaboran** las partes:

1	Usuario hace clic en "Tic Tac Toe" (en el menú)
2	Menú le dice a Navegación: "cambiar a juego 1"
3	Navegación oculta el menú, muestra el juego
4	Motor de Juegos inicializa: puntuación = 0, estado = "jugando"
5	Tic Tac Toe crea tablero vacío, turno = jugador 1
6	UI muestra el tablero

Observa:

- Cada parte hizo SU trabajo
- Nadie hizo el trabajo de otro
- Si mañana cambias el menú, el Tic Tac Toe no se entera

¿Qué pasaría con MAL diseño?

Si todo estuviera en un solo bloque gigante:

```
// ■ TODO MEZCLADO (MAL)
function todaLaApp() {
  // aquí el menú
  // aquí el tic tac toe
  // aquí el ahorcado
  // aquí la navegación
  // aquí las puntuaciones
  // ...
  // 5000 líneas después... ¿dónde estaba el menú?
}
```

Problemas:

- Cambiar el menú rompe los juegos
- Agregar un juego requiere reescribir todo
- Un bug en un juego afecta a otros
- La IA (y tú) se pierden en el código

■ 6. Uso guiado de agentes de IA

¿Qué pedirle a la IA ESTA semana?

Todavía NO le pidas código. Primero valida tu diseño.

■ Prompts buenos:

1. "Tengo este diseño de partes para mi app de mini-juegos: [tu diagrama] ¿Hay alguna responsabilidad que esté mal asignada?"
2. "Explícame qué información debería guardar el 'Motor de Juegos' vs cada juego específico"
3. "Si quiero agregar un tercer juego, ¿qué partes tendría que modificar con este diseño?"

¿Qué revisar OBLIGATORIAMENTE?

Cuando la IA te responda, pregúntate:

1. **¿La IA respeta la separación de responsabilidades?**
Si dice "el menú puede tener lógica de juego" → ■ error
2. **¿La IA sugiere partes demasiado complejas?**
Si inventa 15 partes distintas → ■ overkill
3. **¿La IA explica el PORQUÉ o solo da recetas?**
Si solo da estructura sin justificar → ■■ pide explicación

Errores típicos de la IA

■ Mezclar responsabilidades

"El menú puede guardar el estado del último juego jugado"

Por qué está mal: El menú solo muestra opciones. El estado del juego lo guarda el Motor.

■ Crear dependencias innecesarias

"Cada juego debe conocer al menú para poder volver"

Por qué está mal: Los juegos no necesitan saber que existe un menú. Usan el Sistema de Navegación.

■ Sobre-ingeniería

"Necesitas un Event Bus, un State Manager, un Router..."

Por qué está mal: Con tu nivel, JS puro bien organizado es suficiente. No frameworks todavía.

■ 7. Preguntas de pensamiento crítico

No son preguntas de memoria. Son para **PENSAR**.

Pregunta 1: Responsabilidad cruzada

"Si el menú pudiera iniciar una partida directamente (sin pasar por Navegación), ¿qué problemas causaría?"

Pista: Piensa en qué pasaría si mañana quieres cambiar cómo se navega.

Pregunta 2: Información compartida

"¿Debería el Tic Tac Toe guardar su propia puntuación o debería el Motor de Juegos guardarlo?"

Pista: ¿Qué pasa si quieres mostrar un ranking global de todos los juegos?

Pregunta 3: Escalabilidad

"Si agregas un juego que NO necesita puntuación (ej: un rompecabezas), ¿tu diseño sigue funcionando?"

Pista: ¿Las partes comunes son realmente comunes o son específicas de ciertos juegos?

Pregunta 4: Comunicación

"¿Qué parte debería 'avisar' cuando se ganó un juego?"

Pista: Opciones: A) El juego específico, B) El motor de juegos, C) La UI. Piensa en quién tiene la información necesaria para saberlo.

■ 8. Mini-reto para 1 hora

Objetivo:

Definir con precisión cada parte del sistema SIN escribir código.

Actividad:

Completa esta tabla para cada parte de tu app:

Parte	¿Qué hace?	¿Qué guarda?	¿Con quién habla?	¿Qué NO hace?
Menú Principal				
Motor de Juegos				
Tic Tac Toe				
Navegación				

Criterios de éxito:

- Cada parte tiene 1-3 responsabilidades claras (no 10)
- Ninguna parte hace el trabajo de otra
- Si eliminas una parte, sabes exactamente qué se rompe
- Puedes explicarle esto a alguien sin conocimientos técnicos

Entregable:

1. Tabla completada
2. Diagrama actualizado (puede ser en papel/digital)
3. Responde: "**¿Por qué separar en partes es mejor que tener un solo bloque gigante?**"

■ 9. Qué NO hacer

■ Errores comunes de principiantes:

Las partes se solapan en responsabilidades

- *Ejemplo: El menú Y el motor de juegos guardan puntuaciones*
- **Por qué está mal: Información duplicada = bugs futuros**

Crear partes "porque sí"

- *Ejemplo: Una parte solo para guardar el color del fondo*
- **Por qué está mal: Sobre-ingeniería innecesaria**

No definir cómo se comunican

- *Ejemplo: "Las partes hablan... de alguna forma"*
- **Por qué está mal: Si no defines la comunicación, el código será caótico**

■ Errores típicos de la IA:

Usar nombres vagos

- "Manager", "Handler", "Controller" sin contexto
- Mejor: "Navegación", "Motor de Juegos", nombres concretos

Crear jerarquías complejas

- "AbstractGameFactoryManager extends BaseController"
- Mejor: Estructuras planas y claras

No justificar por qué esa estructura

- "Usa este diseño [código]"
- Mejor: "Usa este diseño porque separa X de Y, lo que permite Z"

■ Reflexión final

Al terminar esta sesión, deberías poder:

- Explicar qué hace cada parte de tu app
- Saber qué información guarda cada parte
- Entender por qué están separadas
- Identificar si una responsabilidad está mal asignada
- Justificar tu diseño ante preguntas críticas

Frase clave de la sesión:

"Un sistema bien diseñado es como un equipo de fútbol: cada jugador tiene su posición y su rol. Si el portero decide hacer de delantero, todo se desmorona."

■ Para la próxima sesión (Sesión 3):

Llevaremos este diseño a HTML/CSS:

- El menú como interfaz visual
- Sin lógica todavía (solo estructura)
- Verás cómo HTML $\neq$ programa

Prepara: Tu tabla de partes y diagrama actualizados, porque los usaremos como mapa.

■ **Recuerda:** No salgas con "más cosas", sal con "una idea clara".

*Si al final de la sesión puedes explicar por qué cada parte hace lo que hace, **misión cumplida.*** ■